CS 216: Bitcoin Transaction Lab Report

Team Members:

- Devanshu Dubey : 240001024
- Pratyush Gupta : 240001054
- Abhinav Patel : 240001004
- Abhay Lodhi : 240001003

GitHub Repository:

https://github.com/DevanshuDubey/CS216-MerkelSecure-bitcoin-transaction-lab

---

```
0s ~ ../CS216-MerkelSecure-bitcoin-transaction-lab ~ master
  • ▶ bitcoind -regtest -daemon
Bitcoin Core starting
0s ~ ../CS216-MerkelSecure-bitcoin-transaction-lab ~ master
  • ▶ python setup_and_fund.py
Connecting to bitcoind...
Attempting to load wallet 'lab_wallet' from disk...
Wallet 'lab_wallet' not found. Creating a new one...
Wallet 'lab_wallet' created.
Mining 101 blocks to mature the coinbase reward (this takes a moment)...

Setup complete! Current Wallet Balance: 50.00000000 BTC
```

Part 1: Legacy Address Transactions (P2PKH)

1.1 Addresses and Funding

Address A: mnJLB9EYuWcnJ4y3qnCgoy1RmYUcW6e1UP
Address B: n2ZmBpJeECGMgpqxUXtPWE8fBbpg3MyuBM
Address C: mkmHGVDzqYcQ8Uh2iToQTdnFJmioinYvnN

1.2 Workflow and Transaction IDs

Transaction A → B TXID:
f20f2ac9b866f04462b5211415d4cd8344a5332a1f8f2cffdebbbd36cd548576
Description: 5.0 BTC was sent from A to B. This created a UTXO (Unspent Transaction Output) at index 0 belonging to Address B.
Transaction B → C TXID:
394d6cdbd64f12f1c0a7333e154d3bfa6ae07030f2b016b0d81092b8f8b2b8a3
Description: This transaction used the UTXO from the previous step as an input to fund a transfer from B to C.

1.3 Decoded Scripts (Analysis)

```
2s ~ ../CS210-MerkelSecure-bitcoin-transaction-lab ~ master
  ● ▶ python part1.py
========== PART 1: LEGACY (P2PKH) ==========

Address A: mnJLB9EYuWcnJ4y3qnCgoy1RmYUcW6e1UP
Address B: n2ZmBpJeECGMgpqxUXtPWE8fBbpg3MyuBM
Address C: mkmHGVDzqYcQ8Uh2iToQTdnFJmioinYvnN

Funding Address A...
Creating Raw Transaction A → B...

--- Decoded Raw TX (A → B) ---
[
  {
    "value": 4.9999,
    "n": 0,
    "scriptPubKey": {
      "asm": "OP_DUP OP_HASH160 e6e29e02fb4f18c2b2e6e09e03528838a03415f8 OP_EQUALVERIFY OP_CHECKSIG",
      "desc": "addr(n2ZmBpJeECGMgpqxUXtPWE8fBbpg3MyuBM)#w9ykpsa5",
      "hex": "76a914e6e29e02fb4f18c2b2e6e09e03528838a03415f888ac",
      "address": "n2ZmBpJeECGMgpqxUXtPWE8fBbpg3MyuBM",
      "type": "pubkeyhash"
    }
  }
]

Broadcasted A → B. TXID: f20f2ac9b866f04462b5211415d4cd8344a5332a1f8f2cffdebbbd36cd548576

Extracting UTXO for Address B...
Found UTXO for B! TXID: f20f2ac9b866f04462b5211415d4cd8344a5332a1f8f2cffdebbbd36cd548576, VOUT: 0
Creating Raw Transaction B → C...
Broadcasted B → C. TXID: 394d6cdbd64f12f1c0a7333e154d3bfa6ae07030f2b016b0d81092b8f8b2b8a3

--- Decoded Signed TX (B → C) showing scriptSig ---
[
  {
    "txid": "f20f2ac9b866f04462b5211415d4cd8344a5332a1f8f2cffdebbbd36cd548576",
    "vout": 0,
    "scriptSig": {
      "asm": "304402204020a8133823e0923436ca5877b72896b310d5ec3ce80b22d1c5f8107222c5c802205526ac62487c7469e37a8184f6ec3f0eb75ff1aa3483554196e1429192c88c24[ALL] 031cafee779bea7125fe6bad90ccc2810be2c936a68d5
8469c2cee4dfee9477b5e",
      "hex": "473044022040208133823e0923436ca5877b72896b310d5ec3ce80b22d1c5f8107222c5c802205526ac62487c7469e37a8184f6ec3f0eb75ff1aa3483554196e1429192c88c240121031cafee779bea7125fe6bad90ccc2810be2c936a68d5
8469c2cee4dfee9477b5e"
    },
    "sequence": 4294967293
  }
]
```

```
--- Size Comparison Metrics (Legacy) ---
Size:   191 bytes
vSize:  191 vbytes
Weight: 764
```

Challenge Script (scriptPubKey from A→B):

"asm": "OP_DUP OP_HASH160 e6e29e02fb4f18c2b2e6e09e03528838a03415f8 OP_EQUALVERIFY OP_CHECKSIG",
"hex": "76a914e6e29e02fb4f18c2b2e6e09e03528838a03415f888ac"
Analysis: This is a P2PKH locking script. It locks the coins to a public key hash (e6e2...). To unlock it, the receiver must provide a public key that hashes to this value and a valid signature.

Response Script (scriptSig from B→C):

"asm":
"304402204020a813...0121031cafee779bea7125fe6bad90ccc2810be2c936a68d58469c2cee4dfee9477b5e",
"hex": "47304402204020a8...77b5e"
Analysis: This script provides the Response to the challenge. It contains the ECDSA Signature and the Public Key of Address B.

1.4 Bitcoin Debugger (btcdeb) Execution

```
● ▶ btcdeb 76a914fcf9b8e6771851f1b57b8ac219de6e5efa96ffe488ac 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841d6270d5b1022042a2befaa2c86effbf0751b94622c327e39e2a56eadec7330b3bae7200dff4c70
1 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
5 op script loaded. type `help` for usage information
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_DUP                                   | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
OP_HASH160                               | 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841 ...
fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4 |
OP_EQUALVERIFY                           |
OP_CHECKSIG                              |
#0000 OP_DUP
btcdeb> step
                <> PUSH stack 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_HASH160                               | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4 | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
OP_EQUALVERIFY                           | 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841 ...
OP_CHECKSIG                              |
#0001 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4 |                                  fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4
OP_EQUALVERIFY                           | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
OP_CHECKSIG                              | 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841 ...
#0002 fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4
btcdeb> step
                <> PUSH stack fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_EQUALVERIFY                           |                                  fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4
OP_CHECKSIG                              |                                  fcf9b8e6771851f1b57b8ac219de6e5efa96ffe4
                                         | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
                                         | 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841 ...
#0003 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
```

```
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
OP_CHECKSIG                              | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
                                         | 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841 ...
#0004 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
error: Signature must be zero for failed CHECK(MULTI)SIG operation
btcdeb> step
script                                   |                                                                  stack
-----------------------------------------+------------------------------------------------------------------
                                         | 02670eddb2cd64555bac23111d96f626e1cdb1d0f71dd50d92cc423d52b6ca2ec0
                                         | 3044022054a70281ba697f341d9a462065466ef4c012f984af3542173a6f841 ...
#0004 OP_CHECKSIG
btcdeb> step
at end of script
btcdeb>
```

The debugger execution demonstrates the successful validation of the script logic. As seen in step #0003, the OP_EQUALVERIFY operator successfully compared our provided Public Key hash against the locking script's required hash. This resulted in a logical 'TRUE' (01) value on the stack, proving we have the correct key to unlock this address.
The script technically stops at the final OP_CHECKSIG operator because btcdeb runs in isolation. OP_CHECKSIG requires the cryptographic hash of the entire transaction (the Sighash) to verify the signature. Since this debugger environment only contains the scripts and not the transaction context, it cannot perform the final math verification. However, the successful execution of the stack up to the signature check provides complete proof that the transaction was constructed and addressed correctly.

Part 2: P2SH-SegWit Address Transactions

2.1 Addresses and Funding

Address A': 2N9CtExWmmX76CqvQs69Y6fB5BdcwuVNq4r
Address B': 2MtqUCHqaY6dFn3eb2jLQ7jTAdqsT5a6t5V
Address C': 2NC6amXBLBY3LhRGQEneSymT4GdsDCuq5S9

2.2 Workflow and Transaction IDs

Transaction A' → B' TXID:
53006c60ac9b9287195b57ff892435c8c974adc74e9b096730172fefcd105b5e
Transaction B' → C' TXID:
1da5bca35e6728cacfe0ac228c9c1b8eb36ac626ed24785c2b43e8a4d51dfd00

2.3 Decoded Scripts (Analysis)

```
0s ~ ··/CS216-MerkelSecure-bitcoin-transaction-lab ~ master
  • ▶ python part2.py
========== PART 2: SEGWIT (P2SH-P2WPKH) ==========

Address A': 2N9CtExWmmX76CqvQs69Y6fB5BdcwuVNq4r
Address B': 2MtqUCHqaY6dFn3eb2jLQ7jTAdqsT5a6t5V
Address C': 2NC6amXBLBY3LhRGQEneSymT46dsDCuq5S9

Funding Address A'...
Creating Raw Transaction A' → B'...

--- Decoded Raw TX (A' → B') ---
[
  {
    "value": 4.9999,
    "n": 0,
    "scriptPubKey": {
      "asm": "OP_HASH160 117220be9058e5f580b90b24baf8ea8e69fc8320 OP_EQUAL",
      "desc": "addr(2MtqUCHqaY6dFn3eb2jLQ7jTAdqsT5a6t5V)#wse2se6c",
      "hex": "a914117220be9058e5f580b90b24baf8ea8e69fc832087",
      "address": "2MtqUCHqaY6dFn3eb2jLQ7jTAdqsT5a6t5V",
      "type": "scripthash"
    }
  }
]

Broadcasted A' → B'. TXID: 53006c60ac9b9287195b57ff892435c8c974adc74e9b096730172fefcd105b5e

Extracting UTXO for Address B'...
Found UTXO for B'! TXID: 53006c60ac9b9287195b57ff892435c8c974adc74e9b096730172fefcd105b5e, VOUT: 0
Creating Raw Transaction B' → C'...
Broadcasted B' → C'. TXID: 1da5bca35e6728cacfe0ac228c9c1b8eb36ac626ed24785c2b43e8a4d51dfd00

--- Decoded Signed TX (B' → C') showing txinwitness ---
[
  {
    "txid": "53006c60ac9b9287195b57ff892435c8c974adc74e9b096730172fefcd105b5e",
    "vout": 0,
    "scriptSig": {
      "asm": "001414938231e8ea1cfae89e263418297f1f10d0dcf1",
      "hex": "16001414938231e8ea1cfae89e263418297f1f10d0dcf1"
    },
    "txinwitness": [
      "304402205768aeff53bbc2dba3565c684723659cb8805e88a2b961e413d3d3a137ebdccf022044aad00cb4e3f0cccd10fc8e1374680faa9a7cdca6d17804cef72254a98a490401",
      "0276a93fd5214d8772d8b03d1eafb84d3f3ec528c5349a812a93ad5be78e931e81"
    ],
    "sequence": 4294967293
  }
]
```

```
--- Size Comparison Metrics ---
Size:   215 bytes
vSize:  134 vbytes
Weight: 533
```

Challenge Script (scriptPubKey from A'→B'):

"asm": "OP_HASH160 117220be9058e5f580b90b24baf8ea8e69fc8320 OP_EQUAL",
"**hex":** "a914117220be9058e5f580b90b24baf8ea8e69fc832087"
Analysis: This is a Pay-To-Script-Hash (P2SH) script. It locks the coins until a redeem script is provided that hashes to 1172....

Response Script (scriptSig and txinwitness from B'→C'):

"scriptSig": { "asm": "001414938231e8ea1cfae89e263418297f1f10d0dcf1" },
"txinwitness": ["304402205768aeff...", "0276a93fd521..."]
Analysis: The scriptSig reveals the inner SegWit program (OP_0 <pubkeyhash>). Crucially, the signature and public key are moved to the txinwitness field, separating them from the main transaction data.

2.4 Bitcoin Debugger (btcdeb) Execution

```
3s ~ ••/CS216-MerkelSecure-bitcoin-transaction-lab ~ master
  ● ▶ btcdeb a914117220be9058e5f580b90b24baf8ea8e69fc832087 001414938231e8ea1cfae89e263418297f1f10d0dcf1
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
6 op script loaded. type `help` for usage information
script                                  |                                        stack
----------------------------------------+----------------------------------------
OP_HASH160                              | 001414938231e8ea1cfae89e263418297f1f10d0dcf1
117220be9058e5f580b90b24baf8ea8e69fc8320 |
OP_EQUAL                                |
<<< P2SH script >>>                     |
0                                       |
14938231e8ea1cfae89e263418297f1f10d0dcf1 |
#0000 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 117220be9058e5f580b90b24baf8ea8e69fc8320
script                                  |                                        stack
----------------------------------------+----------------------------------------
117220be9058e5f580b90b24baf8ea8e69fc8320 |     117220be9058e5f580b90b24baf8ea8e69fc8320
OP_EQUAL                                |
<<< P2SH script >>>                     |
0                                       |
14938231e8ea1cfae89e263418297f1f10d0dcf1 |
#0001 117220be9058e5f580b90b24baf8ea8e69fc8320
btcdeb> step
                <> PUSH stack 117220be9058e5f580b90b24baf8ea8e69fc8320
script                                  |                                        stack
----------------------------------------+----------------------------------------
OP_EQUAL                                |     117220be9058e5f580b90b24baf8ea8e69fc8320
<<< P2SH script >>>                     |     117220be9058e5f580b90b24baf8ea8e69fc8320
0                                       |
14938231e8ea1cfae89e263418297f1f10d0dcf1 |
#0002 OP_EQUAL
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                        stack
----------------------------------------+----------------------------------------
<<< P2SH script >>>                     |                                           01
0                                       |
14938231e8ea1cfae89e263418297f1f10d0dcf1 |
<<< P2SH script >>>
btcdeb>
```

Explanation: The debugger successfully matched the hash of the redeem script (OP_EQUAL), resulting in a 01 (TRUE) result. This proves the P2SH wrapper is valid.

Part 3: Analysis and Explanation

Metric	Legacy(P2PKH)	SegWit(P2SH)
Physical Size	191 bytes	215 bytes
Virtual Size (vSize)	191 vbytes	134 vbytes
Weight	764	533

3.2.1 Comparative Script Structure: Challenge and Response

The fundamental difference between Legacy (Part 1) and SegWit (Part 2) lies in how the "Challenge" (Locking Script) and "Response" (Unlocking Script) are structured and stored.

1. Legacy (P2PKH) Structure

- **Challenge (scriptPubKey):** Uses the OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG template.
- **Response (scriptSig):** The signature and public key are placed directly in the vin field of the transaction.
- **Validation:** The engine concatenates the scriptSig and scriptPubKey to execute on a single stack.

2. SegWit (P2SH-P2WPKH) Structure

- **Challenge (scriptPubKey):** Appears as a standard Pay-to-Script-Hash (P2SH) to remain backward compatible: OP_HASH160 <20-byte-hash> OP_EQUAL.
- **Response (scriptSig & Witness):** The scriptSig contains only a "Redeem Script" (the witness program). The actual cryptographic response (Signature and Public Key) is moved to a separate **Witness** field.
- **Validation:** The witness data is not part of the scriptSig and is instead validated by the SegWit-aware node using the segregated data area.

3.2.2 Analysis of Size and vBytes

The primary difference between Legacy and SegWit transactions is the introduction of Weight and Virtual Size (vSize). In Legacy transactions, every byte of the signature is counted fully toward the size limit.

In SegWit, the signature data is moved to the "Witness" area. The Bitcoin protocol gives a 75% discount to witness data. Every byte in the witness counts as only 1 unit of weight, while non-witness data (the transaction headers and outputs) counts as 4 units.
vSize = Total Weight / 4

Because our SegWit transaction has a weight of 533, its vSize is only 134 vbytes, compared to the Legacy transaction which is 191 vbytes.

3.3.1 Why SegWit Transactions are "Smaller"

SegWit transactions are not necessarily smaller in raw byte count; they are "smaller" in **vBytes**. This is due to the **Witness Discount**:

- Non-witness data (headers, outputs) is "expensive," costing **4 Weight Units** per byte.
- Witness data (signatures) is "cheap," costing only **1 Weight Unit** per byte.
- By moving the bulky signature data to the witness field, the total **Weight** of the transaction is reduced, leading to a lower **vSize**.

3.3.2 Benefits of SegWit Transactions

1. **Lower Transaction Fees:** Since fees are calculated based on vBytes, the 75% discount on witness data allows users to send transactions at a lower cost compared to Legacy formats.
2. **Increased Block Capacity:** By counting witness data as only 1 unit of weight, the effective block capacity is increased beyond 1MB (up to a theoretical 4MB limit), allowing the network to process more transactions per block.
3. **Fixing Transaction Malleability:** In Legacy, the TXID is calculated using the scriptSig. If the signature is changed (malleated), the TXID changes. In SegWit, the signatures are moved to the witness field, which is not part of the TXID calculation. This provides a stable TXID, which is essential for off-chain protocols like the Lightning Network.